

EE105: Introduction to Electrical Engineering

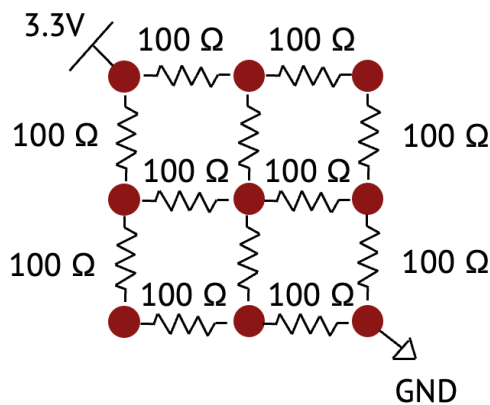
Asymmetric Touchpanel

Objective:

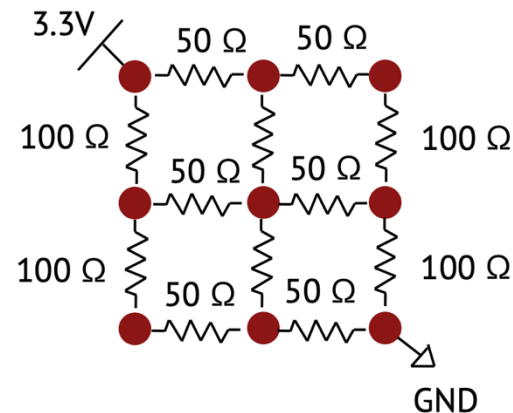
In this lab, we'll learn the basics of Asymmetric Touchpanel, how it works and how it can provide better touch location than the Symmetric Touchpanel. We will also cover analog voltage readout using Arduino and how it can be integrated with sensors.

Symmetric vs Asymmetric Touchpanel:

Due to the asymmetry in the resistance values, the asymmetric touchpanel provides different voltages at each node of the circuit.

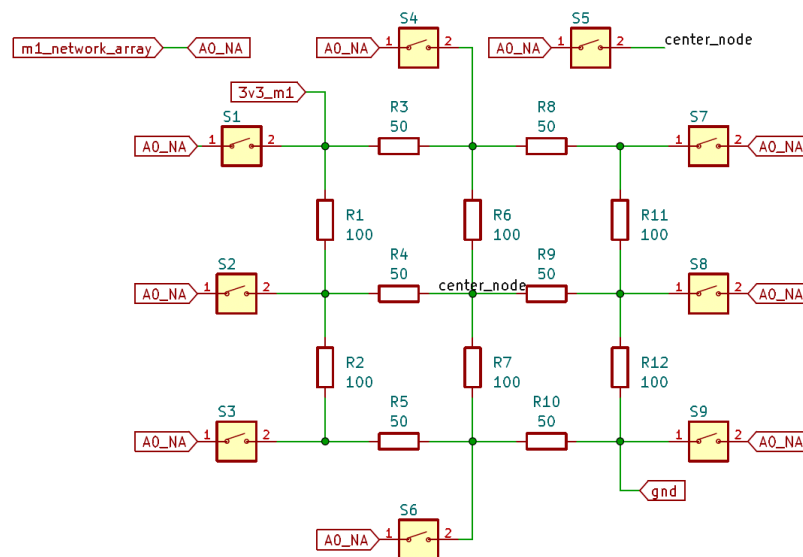


Symmetric Touchpanel



Asymmetric Touchpanel

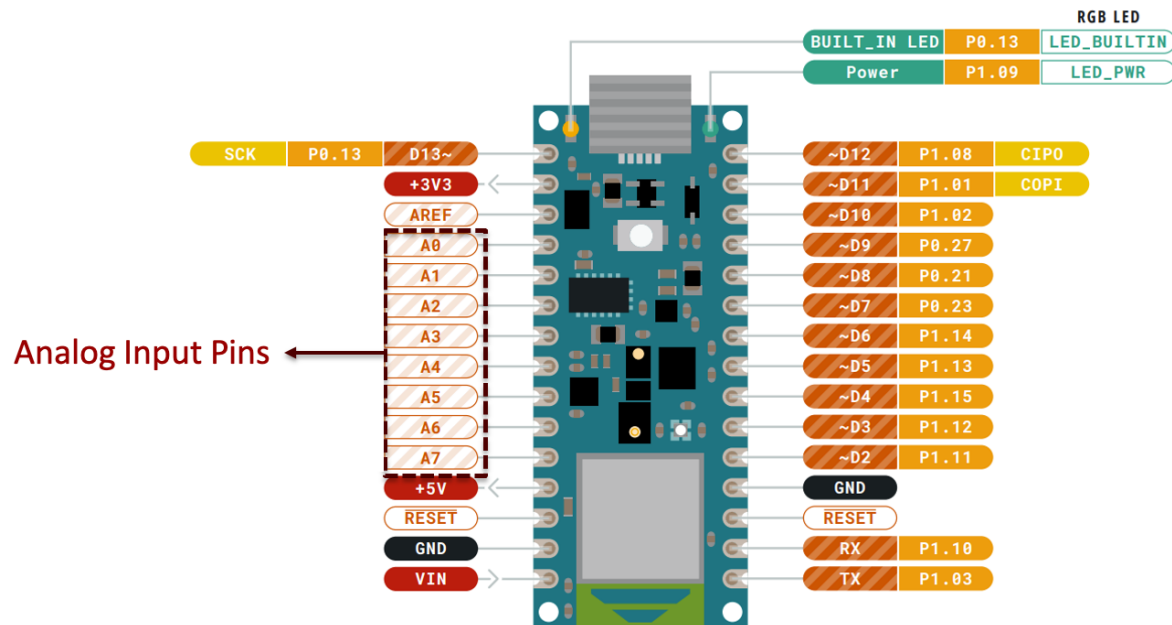
Network-1 of the Dev. Board:



The network-1 of module-1 has switches connected to the analog pin (A0) of the Arduino. When a switch is pressed, the A0 pin gets connected to that corresponding node, allowing us to measure voltage of that node.

Checkpoint-1: Find selector J7 in your dev board and connect the center pin to Module-1 pin.

Analog voltage readout using Arduino Nano 33 BLE Sense:



Arduinos are inherently digital systems. However, they are equipped with analog input pins which can measure analog voltages and convert them to digital signals with the help of built-in ADCs (Analog to Digital Converters).

Read Analog Signal:

Checkpoint-2: Open your Arduino IDE and go to *File > Examples > Basics > AnalogReadSerial*.

A sketch will open which will read analog signal and print the converted digital value in the serial monitor. The example code should look like this:

```
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  // print out the value you read:
  Serial.println(sensorValue);
  delay(1); // delay in between reads for stability
}
```

By pressing the different buttons labeled as S1, S2..., up to S9 in Module 1 you can observe the `sensorValue` value being printed in the serial monitor change.

Checkpoint-3: Find out the **bit count** of ADCs your Arduino boards by looking at the `sensorValue` data in the serial monitor by running the [AnalogReadSerial.ino](#) file.

By default, the **Arduino Nano 33 BLE Sense** uses a lower ADC resolution than it is capable of. This design choice ensures backward compatibility with older boards like the Arduino Uno, which have lower hardware limits. While a higher bit count provides a more accurate digital representation of your analog signal, you must manually unlock this full resolution in your code. You can change the **bit count** of your ADC by using the command ``analogReadResolution(bit)`` within the ``void setup()`` of the Arduino code. The Arduino Nano 33 BLE Sense has a **12-bit ADC**.

Checkpoint-4: Include the ``analogReadResolution(bit)`` function within the ``void setup()`` in the [AnalogReadSerial.ino](#) code and observe the maximum. Use the maximum bit count that your Arduino board can support.

The 3.3 V being provided to the Module 1 Asymmetric Touchpanel Array is given by the second pin on the left side of the Arduino board. Thus, similar to the [AnalogReadSerial.ino](#) file, we can directly read the voltage by making some simple modifications to the code and this can be found in the [ReadAnalogVoltage.ino](#) example file.

Read voltage:

Open your Arduino IDE and go to [File > Examples > Basics > ReadAnalogVoltage](#). A sketch will open which will read analog voltage and print the voltage in the serial monitor. The example code should look like this:

```
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
  float voltage = sensorValue * (5.0 / 1023.0);
  // print out the value you read:
  Serial.println(voltage);
}
```

Checkpoint-5: Make necessary changes to the [ReadAnalogVoltage.ino](#) code to do the following:

1. Utilize the full resolution capacity of your boards ADC to read the analog voltage
2. Normalize the obtained `sensorValue` and map it to the correct voltage level.

Detect which button has been pressed:

The following function outline should allow you to find which button has been pressed based on the detected voltage on the Arduino.

```
void array(){
  int sensorValue = analogRead(A0);
  // float voltage = # Place your code here;
  Serial.print((String)voltage+ " ");
  if (voltage >= ...){Serial.println("S1");}
  else if (voltage < ... && voltage >= ...){Serial.println("S4");}
  else if (voltage < ... && voltage >= ...){Serial.println("S7");}
  else if (voltage < ... && voltage >= ...){Serial.println("S2");}
  else if (voltage < ... && voltage >= ...){Serial.println("S5");}
  else if (voltage < ... && voltage >= ...){Serial.println("S8");}
  else if (voltage < ... && voltage >= ...){Serial.println("S3");}
  else if (voltage < ... && voltage >= ...){Serial.println("S6");}
  else {Serial.println("S9");}
  delay(1000);
}
```

S1	3.30 V
S4	2.49 V
S7	2.11 V
S2	1.95 V
S5	1.65 V
S8	1.35 V
S3	1.18 V
S6	0.80 V
S9	0.00 V

Checkpoint-6:

Create a new Arduino sketch and set the `Serial.begin(9600);` and `analogReadResolution(bit);` within `void setup()`.

Complete the `void array()` function given above and then run this function within the ``void loop()'` of your Arduino code.

Verify that your system finds the correct button whenever it is pressed. If your code doesn't work, compare with the code from **Checkpoint-5**.